secrets: even with a salt the algorithm is still very fast by modern standards. We could, of course, roll our own password hashing, but PHP offers some dedicated functions that are more suited to the task.

password_hash(), password_verify()

The go-to password hashing function is `password_hash()`, which takes the source string and a constant parameter and outputs a formatted string. The output string contains all the information required to repeat the hash: algorithm type, cost, salt, and the hash value itself.

```
print password_hash('weaksauce', PASSWORD_DEFAULT);
```

```
Output:
$2y$10$a6O.WN3IEZ1Bik1Zb7nbiuOjvuL/JCAMwhT45TIoAklR3yl9/nEF6
```

The example above breaks down as follows (note that parts of the output will be random every time because of the salt portion):

- `$2y$` algorithm identifier (in this case, it's the default: Blowfish)
- `10$` cost/complexity
- `a6O.WN3IEZ1Bik1Zb7nbiu` the salt (the Blowfish algorithm generates 22 characters, random every time)
- `OjvuL/JCAMwhT45TIoAklR3yl9/nEF6` the hash (varies with salt)

To verify an incoming password that has been encrypted by `password_hash()`, there's no need to decompose the string like above (though it is entirely possible to do so). `password_verify()` will do the work for you; the first string argument is the user-supplied credential, and the second is a hash string generated by `password_hash()`:

```
$hash = '$2y$10$a6O.WN3IEZ1Bik1Zb7nbiuOjvuL/JCAMwhT45TIoAklR3yl9/nEF6';
var_dump(password_verify('weaksauce', $hash));
```

```
Output:
bool(true)
```